

# GAVEL: Graph World Models for Verified and Efficient Long-Horizon LLM Task Planning

Ruiyang Wang, Hao-Lun Hsu, Swarajh Mehta, Jiwoo Kim, Zhihao Dou, Miroslav Pajic

**Abstract**—Large language models (LLMs) provide a flexible interface for long-horizon robot planning, but generated plans often fail to respect embodiment constraints, recover from planning errors, or reason effectively under partial observability. We present GAVEL, a framework for verifying and repairing long-horizon LLM planning built around an explicit *graph world model*. The graph represents relevant object-relations, action pre-conditions and effects, and probabilistic beliefs over unobserved object locations. This model can predict the consequences of LLM-generated actions before execution, detect violations, and repair those whose corrections follow directly from the world model. This method also reserves LLM replanning solely for errors requiring semantic reasoning. For multi-task instructions, GAVEL reasons over distributions of possible object locations to reorder remaining subtasks and minimize expected search cost. We evaluate GAVEL on BEHAVIOR-1K across 100 single long-horizon tasks and 500 multi-task instructions. With Qwen3-8B, GAVEL improves single-task success from 41.2% to 91.8% and multi-task success from 19.9% to 92.6%. Distributional belief reasoning also reduces travel distance by approximately 5.4% compared with a static variant. These improvements show that an explicit graph world model harness can substantially improve the reliability and efficiency of long-horizon embodied planning across compact local and frontier hosted LLM capabilities.

## I. INTRODUCTION

Large language models (LLMs) offer a promising approach for translating natural-language instructions into a long-horizon sequence of robot actions [1], [2], [3], [4]. However, the resulting plans are not always executable: they may omit prerequisite actions, violate embodiment constraints, or end without reaching the intended state. It was shown that LLMs are unreliable as standalone planners [5], with failure rates that grow with planning horizon, particularly with compact models intended for edge deployment. This motivates pairing language semantics with an explicit model that can verify a plan before execution [6].

Scene-graph methods (e.g., SayPlan [7]) adopt a *propose-and-check* loop: a structured environment model symbolically executes the candidate plan, and infeasible actions are returned to the LLM for another attempt. Yet, recovering from an error does not always require invoking an LLM. For example, when a grasp fails for insufficient proximity, the appropriate recovery action follows directly from the violated precondition. Since the graph that detects the failure already determines its correction, calling an LLM raises cost and risks introducing additional errors.

Such localized repairs are exactly the operations used by EPoG [8]-style planners, which synthesize entire plans from graph edits between the current and goal scene graphs. However, while graph-edit planning is fast and reliable for

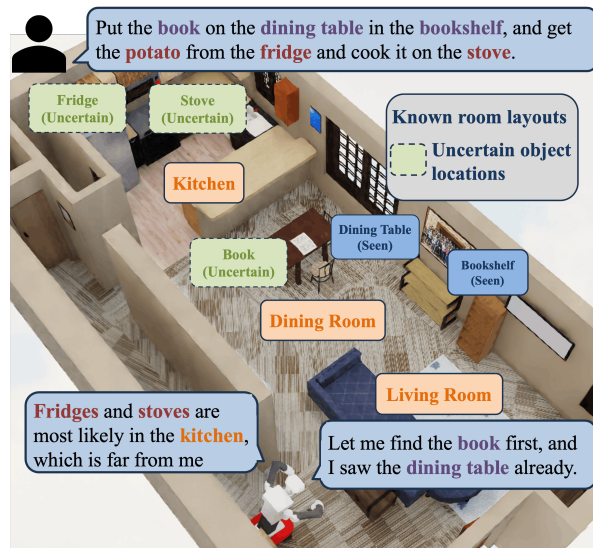


Fig. 1. BEHAVIOR-1K [15] example for long-horizon planning under partial observability. GAVEL verifies and repairs inconsistent LLM plans and uses object-location beliefs to reorder tasks online (i.e., during execution).

spatial rearrangement, it cannot express complex semantic goals (e.g., “wash the plate”) that require a sequence of actions, rather than a simple change in the object relations (e.g., EPoG only supports pick and place).

This motivates treating the graph not as a passive verifier, but as a *world model*: one that simulates the outcome of each action and applies the repairs implied by its modeled state transitions, reserving the LLM call for failures that genuinely require semantic reasoning. Unlike a verifier, such a model could also capture what the robot does not yet know. A robot may know an environment’s layout while remaining uncertain where task-relevant objects are; this matters most when an instruction contains several subtasks, since searching for one object updates the beliefs about the others and can change which task is cheapest to do next. Semantic priors can guide the search for unseen objects [9], [10], [11], [12] and belief-space planning adjusts as observations resolve uncertainty [13], [14], but collapsing each belief to its most likely outcome and fixing task order at initialization leaves this information unused.

Consequently, this work introduces **GAVEL: Graph World Models for Verified and Efficient Long-Horizon LLM Planning**. GAVEL uses one explicit graph world model as the substrate for semantic LLM planning, symbolic consequence reasoning, and probabilistic environment beliefs. Given an LLM-generated plan, GAVEL rolls its actions forward on

the graph, verifies preconditions, goal completion, and safety constraints, and directly repairs model-derivable failures. Under partial observability, the graph maintains distributions over the locations of unseen objects and converts them into expected search and navigation costs. After each completed task, new observations update these beliefs and GAVEL reoptimizes the remaining task order.

Specifically, the main contributions of this work are:

- A graph world model that verifies LLM plans against action preconditions, goal completion, and safety. It repairs the failures its own action semantics determine, and passes only semantic failures back to the LLM.
- An extension to partially observed multi-task execution that retains full distributions over unobserved object locations, converts them into expected search cost, and re-optimizes the remaining task order online.
- An evaluation on BEHAVIOR-1K [15] across single long-horizon tasks, and partially observed multi-task instructions. On tests with both local and hosted LLMs, we demonstrate substantial improvements in task success and execution efficiency, and show that explicit world-model reasoning remains complementary to increasing LLM capability.

## II. RELATED WORK

### A. LLMs for Embodied Task Planning

LLMs are increasingly used as high-level planners for embodied agents. SayCan [1] scores candidate actions by learned affordances, Code as Policies [3] and Prog-Prompt [16] have the LLM write executable policies directly, and LLM-Planner [4] and Inner Monologue [2] replan from observed or reported feedback. A second group pairs the LLM with explicit solvers instead. LLM+P [17] formulates the problem for a classical planner, while PRoC3S [18] and Text2Motion [19] check constraint satisfaction before execution. The prior works consistently show that an explicit model’s feedback improves the reliability of task planning.

### B. Scene-Graph Grounding and Verification

Graph structures have recently been explored as explicit world representations for agent reasoning, from a general Graph World Model for structured and multimodal prediction [20] to AriGraph [21], which learns a knowledge-graph world model with semantic and episodic memory for LLM agents. In robotics, scene graphs offer a natural planning substrate by grounding objects, spatial relations, and states directly [22], [23]. SayPlan [7] and VeriGraph [24] use them only for pre-execution verification, and LookPlanGraph [25] refines them online via VLMs. GAVEL instead treats the graph as a full world model, representing action transitions and uncertainty for subsequent planning.

### C. Graph Planning under Partial Observability

Scene graphs can serve as persistent representations as an environment is incrementally observed [26], [27], [28]. When task-relevant objects remain unseen, semantic knowledge can guide where to search. PONI [9] learns semantic potential

functions over the unexplored space, SEEK [11] combines a dynamic scene graph with a Relational Semantic Network (RSN) to predict likely object locations, and L3MVN [10] and COMRES-VLM [12] use LLM/VLM commonsense to guide exploration. GAVEL follows SEEK’s RSN formulation for initializing room-location beliefs, while retaining the resulting distribution for downstream planning.

EPoG [8] is the closest baseline to GAVEL under partial observability. It maintains a belief graph of observed and predicted objects and constructs the global plan directly from graph edits toward the goal, using an LLM mainly for situated local replanning as observations arrive. GAVEL reverses this division of responsibility: the LLM generates the semantic procedure, while the graph acts as an explicit *world model* that rolls the plan forward, detects violations, and directly repairs failures whose corrections are implied by the modeled action semantics. Only failures requiring additional semantic reasoning are returned to the LLM. GAVEL further retains full distributions over unobserved object locations and uses them to estimate execution costs and reoptimize task order online. This connects GAVEL to prior work on belief-space replanning [13], [14] and multi-task subgoal selection [29].

## III. PROBLEM DEFINITION

### A. Problem Statement

Consider a robot in an indoor environment with room instances  $\mathcal{R}$  of semantic type  $\tau : \mathcal{R} \rightarrow \mathcal{T}$ , where  $\mathcal{T}$  is the set of semantic types (e.g., {kitchen, dining room, etc.}), and task-relevant objects  $\mathcal{O}$  with categories  $c(o)$ . We represent the environment by a typed scene graph defined as:

$$G = (V, E, \Phi), \quad V = \mathcal{R} \cup \mathcal{O} \cup \{\rho\}, \quad (1)$$

where  $\rho$  is the robot,  $E$  represents relations including {*near*, *under*, *room\_connect*, *room\_inside*, *object\_inside*, *on\_top*, *next\_to*, *holding*}, and  $\Phi$  contains the unary flags {open, toggled, cooked, washed, dried}. We distinguish the hidden physical state  $G_t^*$  from the robot’s believed graph  $G_t$ , for execution step  $t \in \{0, \dots, H-1\}$ .

We assume that the room layout  $(\mathcal{R}, \tau)$  and its connectivity are known, and object relations specified by the instruction or already observed are represented deterministically in  $G_t$ . The room locations of remaining unobserved objects are uncertain. Let  $x_t \in \mathcal{R}$  denote the robot’s room at step  $t$ .

The robot acts through grounded primitives  $a_t = (\alpha, \xi) \in \mathcal{A}$ , where  $\alpha$  is the action type and  $\xi$  its argument. Each primitive has graph-valued preconditions and effects  $\langle \text{pre}_a, \text{eff}_a \rangle$  defining the transition model

$$f(G, a) = \begin{cases} \text{eff}_a(G), & \text{if } \text{pre}_a(G) \text{ holds,} \\ \perp, & \text{otherwise,} \end{cases} \quad (2)$$

where  $\perp$  denotes an invalid transition. A plan  $\pi = \langle a_0, \dots, a_{H-1} \rangle$  is *applicable* from  $G_0$  if every action satisfies its preconditions along the induced trace. GAVEL uses (2) as its predictive world model, while physical execution evolves the hidden state  $G_t^*$ . The environment is partially

observed, so we maintain a belief over the room locations of uncertain objects,

$$B_t = \{b_o^t\}_{o \in \mathcal{O}}, \quad b_o^t(r) = P(\text{room}(o) = r \mid \mathcal{H}_t), \quad (3)$$

where  $\mathcal{H}_t$  is the history of actions and observations up to step  $t$ . The observation at each step,  $z_t$ , is updated through the sensor model  $\Xi$  as:  $z_t = \Xi(x_t, G_t^*)$ .

For each uncertain object, the observation either localizes it, if present, or prunes the observed room from its candidate set if absent. The belief is updated and actions are selected by an execution policy  $\mu$  from the belief state,

$$B_{t+1} = \Upsilon(B_t, a_t, z_{t+1}), \quad a_t = \mu(G_t, B_t),$$

while  $G_t^*$  is revealed only through observations.

An instruction  $\mathcal{I}$  is decomposed into  $N$  tasks  $\{\mathcal{I}_i\}_{i=1}^N$  with partial-graph goals  $\{g_i\}_{i=1}^N$ , each specifying relations and unary states that must hold at termination. The overall goal is  $g = \bigcup_{i=1}^N g_i$ , satisfied when  $\text{unmet}(g, G) = \emptyset$ . We additionally impose a trace-level safety predicate  $\text{safe}(a_{0:H-1})$ , capturing restoration requirements such as closing opened containers and switching off safety-critical appliances.

We consider multi-task instructions whose tasks are independent by construction. Their goal-object sets are disjoint,  $\text{obj}(g_i) \cap \text{obj}(g_j) = \emptyset$  for  $i \neq j$ , so every ordering is admissible and ordering affects execution cost rather than goal feasibility. Generated subplans may nevertheless interact through auxiliary objects or state changes, motivating explicit validation of their composition.

**Problem: Verified Multi-Task Planning under Partial Observability.** Given an instruction  $\mathcal{I}$ , an initial believed graph  $G_0$  with belief  $B_0$ , and an unknown initial physical state  $G_0^*$  consistent with  $B_0$ , find a policy  $\mu \in \mathcal{M}$  minimizing the expected physical execution distance, where  $\mathcal{M}$  denotes the set of admissible policies of the form  $\mu(G_t, B_t) \in \mathcal{A}$ :

$$\mu^* = \arg \min_{\mu \in \mathcal{M}} \mathbb{E}_{G_0^* \sim B_0, \mu} [C_{\text{exec}}(\mu)]. \quad (4)$$

where

$$C_{\text{exec}}(\mu) = \sum_{t=0}^{H-1} \ell(a_t; G_t^*), \quad (5)$$

subject to

$$\begin{aligned} \text{unmet}(g, G_H^*) &= \emptyset, & \text{safe}(a_{0:H-1}) &\text{ holds,} \\ \text{pre}_{a_t}(G_t^*) &\text{ holds,} & G_{t+1}^* &= f(G_t^*, a_t), \\ a_t &= \mu(G_t, B_t), & z_{t+1} &= \Xi(x_{t+1}, G_{t+1}^*), \\ B_{t+1} &= \Upsilon(B_t, a_t, z_{t+1}). \end{aligned}$$

Here  $\ell(a_t; G_t^*)$  is the physical navigation distance incurred by  $a_t$ , and the constraints hold for all  $t \in \{0, \dots, H-1\}$ .

Problem III-A couples two difficulties. First, an LLM-generated plan is not guaranteed to satisfy the transition constraints in (2), so executability must be checked explicitly. Second, execution cost depends on uncertain object locations and changes as observations update  $B_t$ . GAVEL addresses both through a common graph world model: explicit action semantics support verification and repair, while the evolving belief supports uncertainty-aware task ordering.

## B. Belief-Aware Task Ordering

For a multi-task instruction, GAVEL selects remaining task execution order to minimize expected travel under current belief  $B_t$ . For an unlocalized object  $o$ , let  $(r_1, \dots, r_K)$  denote its reachable candidate rooms, ordered by decreasing  $b_o^t(r)$  where shortest-path distance breaks ties. If  $o$  is found in the  $k$ -th searched room, the incurred search cost is

$$\kappa_k = D(x_t, r_1) + \sum_{m=1}^{k-1} [s(r_m) + D(r_m, r_{m+1})] + \eta s(r_k),$$

where  $D(\cdot, \cdot)$  is the shortest-path distance,  $s(r) = A_{\text{trav}}(r)/w_{\text{cov}}$  approximates the full search cost of room  $r$ ,  $A_{\text{trav}}(r)$  is the traversable area of room  $r$ ,  $w_{\text{cov}}$  is the effective sensor coverage width, and  $\eta \in [0, 1]$  is the expected fraction searched before detection. The expected cost of reaching an unlocalized object is

$$C_{\text{srch}}(x_t, o; B_t) = \frac{1}{Z_t} \sum_{k=1}^K b_o^t(r_k) \kappa_k, \quad Z_t = \sum_{k=1}^K b_o^t(r_k).$$

If  $o$  is localized, we use its geometric navigation cost,

$$C_{\text{loc}}(q_t, o) = d_{\text{nav}}(q_t, p_o), \quad (6)$$

where  $q_t$  and  $p_o$  denote the robot and object positions. These costs are used during belief-conditioned rollouts to estimate the cost of executing each task and transitioning between.

Specifically, let  $h[j]$  denote the expected cost of executing task  $j$  first from the current state  $(x_t, B_t)$ , and let  $A[i, j]$  denote the expected cost of executing task  $j$  after task  $i$ . For an ordering  $\varsigma = (\varsigma_1, \dots, \varsigma_N)$ , we approximate its expected execution cost by

$$J(\varsigma) = h[\varsigma_1] + \sum_{p=2}^N A[\varsigma_{p-1}, \varsigma_p]. \quad (7)$$

Constructing  $(h, A)$  requires  $\mathcal{O}(N^2)$  belief-conditioned rollouts. GAVEL executes only the first task of the minimum-cost ordering, updates  $B_t$  using the resulting observations, and recomputes the ordering for the remaining tasks. Since  $N \leq 5$  in our experiments, all candidate permutations are enumerated and Eq. (7) is minimized exactly.

## IV. METHOD

### A. System Overview

GAVEL combines semantic LLM planning with an explicit graph world model for verified, belief-aware long-horizon execution. As shown in Fig. 2 and Alg. 1, an instruction  $\mathcal{I}$  is first decomposed into tasks  $\{\mathcal{I}_i\}_{i=1}^N$ . Task understanding extracts the entities, relations, and goals needed to initialize the graph, while the RSN assigns room-location beliefs to uncertain objects.

A single LLM query then produces an action plan  $\pi_i$  per task. The graph world model rolls each plan forward, repairs model-derivable failures, and returns unresolved violations to the LLM at most  $T$  (e.g., 5) times. Once verified, GAVEL orders the plans by the cost in Eq. (7) under the current belief. After each completed task, new observations update the belief and the remaining order is reoptimized.

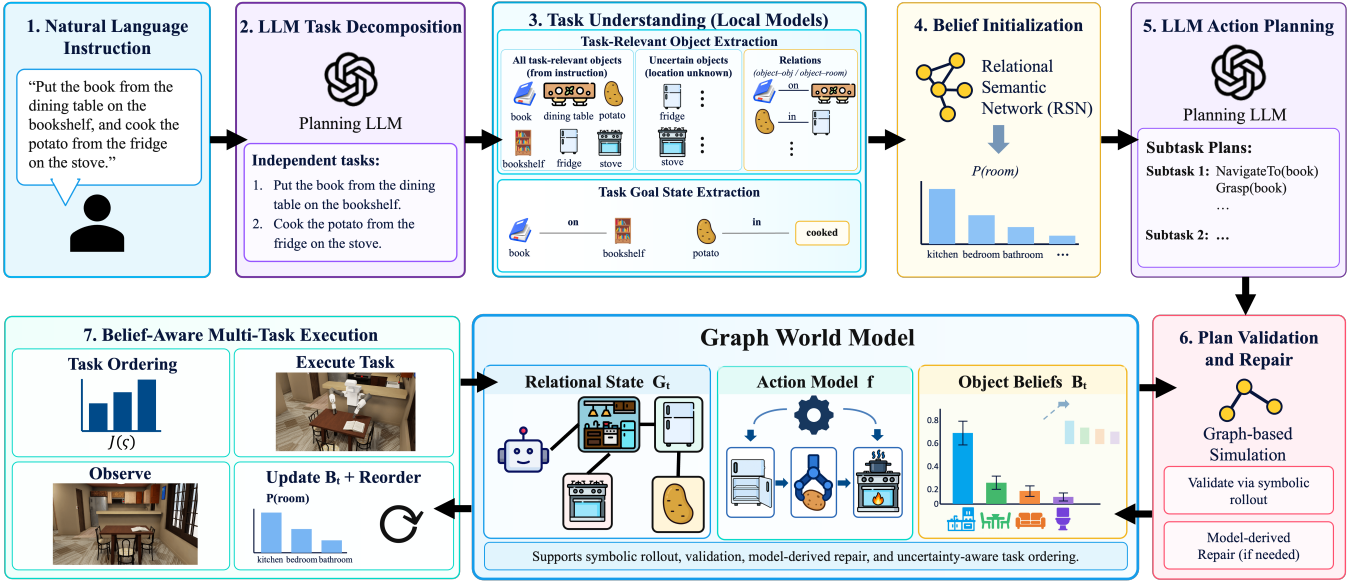


Fig. 2. Overview of the GAVEL pipeline. The instruction is decomposed into tasks, objects, and extracted goals. Uncertain objects receive RSN predicted location beliefs. An LLM query generates task plans, which the graph world model verifies and repairs before execution. GAVEL then updates beliefs from observations and reorders the remaining tasks online.

### B. Task Understanding

For each decomposed task  $\mathcal{I}_i$ , GAVEL extracts (i) task-relevant objects and relations to initialize the belief graph and (ii) goal states used to evaluate task completion (Fig. 2). Both mappings use lightweight LoRA adapters of Qwen3-1.7B. These are shared by all planner models and baselines.

*a) Task-relevant object extraction:* The grounding adapter maps  $\mathcal{I}_i$  to an object set  $\mathcal{O}_i$  and explicitly stated relations  $\mathcal{E}_i$ , which are inserted directly into the believed graph rather than inferred from semantic priors. For example, “get the potato from the fridge” yields  $\text{inside}(\text{potato}, \text{fridge})$ , localizing the potato through the fridge. Only objects whose locations remain uncertain are passed to the RSN:

$$\mathcal{O}_i^{\text{qry}} = \{o \in \mathcal{O}_i : \text{room}(o) \text{ is uncertain in } \mathcal{E}_i\}, \quad (8)$$

*b) Task-goal extraction:* A second adapter maps  $\mathcal{I}_i$  to a partial-graph  $g_i$  describing the goal terminal state. It contains spatial predicates like  $\text{on\_top}(o_1, o_2)$  and  $\text{inside}(o_1, o_2)$ , and semantic states like  $\text{washed}(o)$ , and  $\text{dried}(o)$ .

Both adapters operate on natural-language instruction and are trained on 8,000 synthetic instruction–target pairs with a disjoint 500-instruction validation set.

### C. Belief Initialization

For each uncertain object  $o \in \bigcup_i \mathcal{O}_i^{\text{qry}}$ , GAVEL initializes a room-location belief using a Relational Semantic Network (RSN), following SEEK [11]. A frozen text encoder  $\phi(o) \in \mathbb{R}^{384}$  followed by an MLP  $f_\theta$  predicts a score for each room type  $\tau \in \mathcal{T}$ :

$$o \xrightarrow{\phi} \phi(o) \xrightarrow{f_\theta} \{\tilde{f}_o(\tau)\}_{\tau \in \mathcal{T}}. \quad (9)$$

Because the RSN predicts room types while GAVEL reasons over room instances, each type score is distributed uniformly among reachable instances of that type,

$$b_o^0(r) \propto \frac{\tilde{f}_o(\tau(r))}{|\{r' \in \mathcal{R}^{\text{re}} : \tau(r') = \tau(r)\}|}. \quad (10)$$

The distribution is then normalized over reachable rooms.

We use BAAI/bge-small-en-v1.5 as the frozen encoder and a three-layer MLP with hidden dimensions 256–128–64 and dropout 0.2. The RSN is trained from scene object placements using weighted binary cross-entropy and calibrated using Platt scaling.

### D. LLM Action Planning

After task understanding and belief initialization, the planner receives the instruction  $\mathcal{I}$ , believed graph  $G_0$ , predicted object-location beliefs  $B_0$ , task goals  $\{g_i\}_{i=1}^N$ , and the available action definitions. As shown in Alg. 1, a single LLM query produces one grounded action sequence for each decomposed task,

$$\{\pi_i\}_{i=1}^N = \text{LLM}(\mathcal{I}, G_0, B_0, \{g_i\}_{i=1}^N). \quad (11)$$

Each resulting  $\pi_i$  is then passed to the graph world model for validation and repair. If a violation cannot be resolved from the modeled action semantics, its structured failure description is returned to the LLM for another planning attempt, up to a budget number of calls  $T$ .

### E. Graph World Model

The graph world model provides the symbolic transition used to predict the consequences of LLM-generated actions. GAVEL operates over nine grounded primitives,

$$\mathcal{A} = \{\text{NAVIGATE\_TO}, \text{GRASP}, \text{RELEASE}, \text{PLACE\_ON\_TOP}, \text{PLACE\_INSIDE}, \text{OPEN}, \text{CLOSE}, \text{TOGGLE\_ON}, \text{TOGGLE\_OFF}\}.$$

---

**Algorithm 1** GAVEL

---

**Require:** instruction  $\mathcal{I}$ ; believed graph  $G_0$ ; LLM budget  $T$ 

```

1:  $\{\mathcal{I}_i\}_{i=1}^N \leftarrow \text{DECOMPOSE}(\mathcal{I})$ 
2: for  $i = 1$  to  $N$  do
3:    $(\mathcal{O}_i, \mathcal{E}_i) \leftarrow \text{EXTRACTOBJECTS}(\mathcal{I}_i)$ 
4:    $\mathcal{O}_i^{\text{qry}} \leftarrow \text{UNCERTAIN}(\mathcal{O}_i, \mathcal{E}_i)$ 
5:    $g_i \leftarrow \text{PARSEGOAL}(\mathcal{I}_i)$     $G_0 \leftarrow \text{INSERT}(G_0, \mathcal{E}_i)$ 
6: end for
7:  $B_0 \leftarrow \text{RSN}(\bigcup_i \mathcal{O}_i^{\text{qry}}, G_0)$ 
8:  $p \leftarrow \text{PROMPT}(\mathcal{I}, G_0, B_0, \{g_i\})$ 
9: for  $iter = 1$  to  $T$  do
10:   $\{\pi_i\}_{i=1}^N \leftarrow \text{PARSE}(\text{LLM}(p))$ 
11:  for  $i = 1$  to  $N$  do
12:     $\pi_i \leftarrow \text{REPAIR}(G_0, \pi_i, g_i)$ 
13:     $\omega_i \leftarrow \text{VALIDATE}(G_0, \pi_i, g_i)$ 
14:  end for
15:  if all  $\omega_i$  are valid then
16:    break
17:  end if
18:   $p \leftarrow \text{COMPLAINT}(\mathcal{I}, G_0, \{\pi_i, \omega_i\})$ 
19: end for
20:  $\mathcal{U} \leftarrow \{1, \dots, N\}$ ,    $(G, B) \leftarrow (G_0, B_0)$ 
21: while  $\mathcal{U} \neq \emptyset$  do
22:   $(h, A) \leftarrow \text{ROLLOUT}(\mathcal{U}, B, x)$ 
23:   $\Sigma \leftarrow \text{SORT}_J(\text{Perm}(\mathcal{U}))$ 
24:  for  $\varsigma \in \Sigma$  do
25:     $\omega \leftarrow \text{VALIDATE}\left(G, \bigoplus_{j=1}^{|\mathcal{U}|} \pi_{\varsigma_j}, \bigcup_{i \in \mathcal{U}} g_i\right)$ 
26:    if  $\omega$  is valid then
27:      break
28:    end if
29:  end for
30:   $(G, z) \leftarrow \text{EXECUTE}(\pi_{\varsigma_1})$ 
31:   $B \leftarrow \Upsilon(B, \pi_{\varsigma_1}, z)$     $\mathcal{U} \leftarrow \mathcal{U} \setminus \{\varsigma_1\}$ 
32: end while

```

---

Each grounded action  $a = (\alpha, \xi)$  has modeled preconditions and effects  $\langle \text{pre}_a, \text{eff}_a \rangle$ , defining the transition in Eq. (2).

The preconditions capture four forms of executability: *proximity*, requiring the robot to be near the interaction target; *affordance*, requiring the target to support the requested action; *gripper state*, enforcing holding constraints; and *accessibility*, requiring enclosing containers to be open. NAVIGATETo establishes proximity but cannot target an object hidden inside a closed container.

Action effects update graph relations and object states. GRASP establishes the holding relation, placement transfers the held object to its destination, and appliance operation can induce semantic states such as cooked, washed, or dried. GAVEL also tracks restoration constraints: containers opened during a plan must be closed, and safety-critical appliances switched on must later be switched off.

#### F. Plan Validation and Repair

Given a candidate plan  $\pi_i = \langle a_0, \dots, a_{H-1} \rangle$ , GAVEL predicts its consequences by rolling the actions forward on the graph world model. Each action is applied only if

---

**Algorithm 2** REPAIR: Model-derived plan repair

---

**Require:** believed graph  $G$ , plan  $\pi$ , goal  $g$ 

```

1:  $\pi^* \leftarrow \pi$     $\omega^* \leftarrow \text{VALIDATE}(G, \pi^*, g)$     $\mathcal{S} \leftarrow \{\pi\}$ 
2: loop
3:    $\omega \leftarrow \text{VALIDATE}(G, \pi, g)$ 
4:   if  $\text{rk}(\omega, \pi) > \text{rk}(\omega^*, \pi^*)$  then
5:      $(\pi^*, \omega^*) \leftarrow (\pi, \omega)$ 
6:   end if
7:   if  $\omega.\text{app}$  and  $\omega.\text{safe}$  and  $\omega.\text{unmet} = \emptyset$  then
8:     return  $\pi$ 
9:   end if
10:  if no model-derived edit exists for  $\omega$  then
11:    break
12:  end if
13:   $\pi' \leftarrow \text{EDIT}(\pi, \omega, g)$ 
14:  if  $\pi' \in \mathcal{S}$  then
15:    break
16:  end if
17:   $\mathcal{S} \leftarrow \mathcal{S} \cup \{\pi'\}$     $\pi \leftarrow \pi'$ 
18: end loop
19: return  $\pi^*$ 

```

---

its preconditions hold. Otherwise rollout stops at the first violation. The resulting graph is then checked against goal  $g_i$  and trace-level safety constraints. A plan is valid iff

$$\text{unmet}(g_i, G_H) = \emptyset \quad \text{and} \quad \text{safe}(\pi_i). \quad (12)$$

The validator returns a structured verdict  $\omega$  identifying applicability failures, unmet goal conditions, and safety violations.

Many such failures imply their own corrections. As summarized in Alg. 2, REPAIR recursively applies edits whose effects follow directly from the action model: a failed grasp due to missing proximity induces NAVIGATETo, and an object inside a closed container induces navigation to the container followed by OPEN. Safety violations similarly induce restoration actions. When no model-derived repair exists, the unresolved verdict is returned to the LLM as structured feedback. The visited set  $\mathcal{S}$  prevents cyclic repair by ending when an edit leads to a previously considered plan.

Because successive edits may not monotonically improve a plan, GAVEL retains the best previous candidate:

$$\text{rk}(\omega, \pi) = (\mathbf{1}[\omega.\text{app}], \mathbf{1}[\omega.\text{unmet} = \emptyset], \mathbf{1}[\omega.\text{safe}], -\omega.d).$$

where  $\omega.d$  defines the execution cost of the plan. Thus, the budget  $T$  limits LLM generations, while graph-derived repair may perform multiple edits between two LLM queries.

#### G. Belief-Aware Multi-Task Execution

Once the task plans are repaired and verified, GAVEL determines which remaining task to execute next under current belief  $B_t$ . At each task boundary, ROLLOUT constructs the first-task costs  $h$  and pairwise transition costs  $A$  using the cost model in Sec. III-B. Eq. (7) evaluates every permutation of the remaining task set  $\mathcal{U}$ . GAVEL sorts these orderings by increasing expected cost and validates each one's concatenated plans against the union of the corresponding goals,

selecting the first that passes. If the cheapest ordering causes cross-task interference, the next is considered instead.

Only the first task of the selected ordering is executed. The resulting observations update the beliefs of all uncertain objects: observed objects become localized, while searched rooms in which an object is not seen are removed from its belief support. GAVEL then rebuilds  $(h, A)$  and reoptimizes the remaining order, so information gathered during one task immediately affects the schedule of those that follow.

## V. EXPERIMENTAL EVALUATION

### A. Simulation Setup

We evaluate GAVEL at the symbolic action level of BEHAVIOR-1K, as our focus is high-level task planning rather than low-level manipulation. For large-scale evaluation, we implement the BEHAVIOR-1K action transitions in a lightweight 2-D executor retaining the original floor plans, object properties, task predicates, and navigation geometry. Navigation is geometric: traversability maps are eroded by the robot base radius and paths are planned with  $A^*$ . Unseen objects are searched through frontier-based exploration with a wedge-shaped camera. A completed room search localizes observed objects and rules that room out for unobserved ones. For the ordering cost model, we use  $w_{\text{cov}} = 1.2\text{m}$ . The action space is unchanged: all nine primitives map one-to-one onto the BEHAVIOR-1K/OmniGibson interface, and every benchmark task is directly executable in OmniGibson. To validate the abstraction, we ran all 100 single-task instructions in OmniGibson; success outcomes agreed with the executor on every instance. The executor reduces execution time from roughly 17 minutes per plan to about one second, enabling 5,500 evaluations at scale. The supplementary video shows GAVEL executing example tasks in OmniGibson.

We measure *Success* with the executor after the plan is simulated, *Distance* as how far the robot travels, and *Planning time* as decomposition, LLM calls, repair, and ordering, measured on an idle GPU. Results are averaged over five random seeds and reported as mean  $\pm$  std.

### B. Benchmarks

Prior work in this setting typically evaluates on relatively small task sets: five object-transport tasks across 46 scenes in EPoG [8], and 90 instructions in SayPlan [7]. We follow the same evaluation paradigm at substantially larger scale, first verifying every reference plan in our executor so measured failures reflect planning rather than invalid specifications.

The **single-task benchmark** holds 100 long-horizon tasks across 10 scenes: fetching objects, opening containers, operating appliances, and delivering things to a specified destination. Reference plans average 12.2 primitives (8–17) and collectively include 240 goal predicates. The **multi-task benchmark** holds 500 instructions over the scenes, each with  $N \in \{2, 3, 4, 5\}$  independent tasks, averaging 18.8 primitives across 4.66 rooms. Of these, 199 need a semantic state that only operating an appliance can produce.

TABLE I  
EXPERIMENT 1: 100 SINGLE LONG-HORIZON TASKS.

| Model    | Method      | Success (%)                      | Average Attempt | Time (s)        |
|----------|-------------|----------------------------------|-----------------|-----------------|
| Qwen3-4B | llm-only    | 21.8 $\pm$ 1.5                   | 1.00 $\pm$ 0.0  | 9.2 $\pm$ 1.0   |
| Qwen3-4B | sayplan     | 55.4 $\pm$ 1.9                   | 3.4 $\pm$ 1.7   | 27.6 $\pm$ 12.0 |
| Qwen3-4B | gavel-basic | 67.7 $\pm$ 1.6                   | 1.00 $\pm$ 0.0  | 9.2 $\pm$ 1.0   |
| Qwen3-4B | gavel       | <b>88.8 <math>\pm</math> 1.6</b> | 1.7 $\pm$ 1.3   | 15.0 $\pm$ 11.1 |
| Qwen3-8B | llm-only    | 41.2 $\pm$ 1.2                   | 1.00 $\pm$ 0.0  | 13.3 $\pm$ 6.4  |
| Qwen3-8B | sayplan     | 76.4 $\pm$ 1.5                   | 2.7 $\pm$ 1.7   | 24.6 $\pm$ 15.8 |
| Qwen3-8B | gavel-basic | 76.2 $\pm$ 1.2                   | 1.00 $\pm$ 0.0  | 13.5 $\pm$ 6.4  |
| Qwen3-8B | gavel       | <b>91.8 <math>\pm</math> 1.3</b> | 1.5 $\pm$ 1.1   | 15.5 $\pm$ 9.4  |

### C. Compared Methods

Every method sees the same initial state, LLM budget, and believed graph; the SayPlan and EPoG baselines reproduce their relevant mechanism inside this shared pipeline.

`llm-only` generates one plan and runs it. `sayplan` adds the validation-feedback loop of [7] with up to  $T$  LLM re-calls, isolating *feedback without repair*. `epog` derives actions from the difference between the believed and goal graphs, ordered by travel cost [8], isolating planning from graph-state differences alone. `oracle` uses ground-truth locations and reference subplans as a cost reference.

Among our variants, `gavel-basic` takes one LLM proposal and then relies only on graph repair, isolating *repair without feedback*. `gavel-map` commits each uncertain object to its most likely room and fixes the order up front; `gavel-static` keeps the full distribution but still fixes the order. Finally, `gavel` keeps the distribution and reorders after every finished task.

### D. Belief and Cost-Model Quality

The RSN trains on 11,218 placements across 51 scenes, 197 object categories, and 37 room types, with scenes held out. It reaches an AUC of 0.904 and a Brier score of 0.057. Still, the most likely room is a poor summary of that belief: across 832 held-out queries the true room ranks first only 47% of the time, so collapsing  $b_o$  to  $\hat{r}_o$  discards information.

The pairwise cost model tracks what the robot actually drives, at  $R^2 = 0.93$  for four-task and  $R^2 = 0.89$  for five-task instructions, which is what justifies the  $\mathcal{O}(N^2)$  construction instead of rolling out every permutation.

### E. Experiment 1: Single Long-Horizon Task Reliability

We first ask whether an LLM can reliably generate a complete long-horizon plan before considering partial observability or multi-task scheduling. As shown in Table I, **LLM-only** planning is highly unreliable for the compact models: Qwen3-4B and Qwen3-8B succeed on only 21.8% and 41.2% of tasks, confirming that simply generating a plan and executing it is insufficient at these horizons.

A common alternative is to detect an invalid plan and return the failure to the LLM for replanning, as in SayPlan and related feedback-based approaches [7], [2]. This improves success to 55.4% and 76.4%, but requires 3.4 and 2.7 LLM calls per task. More importantly, feedback does not guarantee that a compact LLM will correctly repair the reported error.

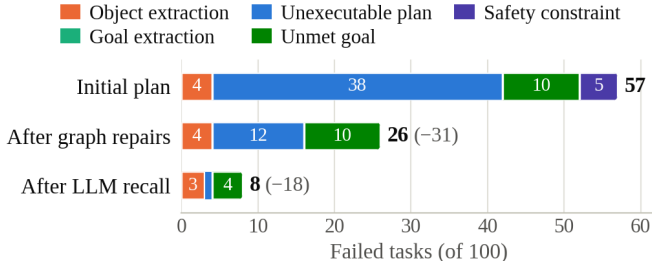


Fig. 3. Failure breakdown for Qwen3-8B GAVEL in Experiment 1. The figure shows remaining failures after each pipeline step.

TABLE II  
EXPERIMENT 2: 500 MULTI-TASK INSTRUCTIONS USING QWEN3-8B.

| Method       | Success (%)                      | Distance (m)                       | Time (s)        |
|--------------|----------------------------------|------------------------------------|-----------------|
| llm-only     | 19.9 $\pm$ 0.7                   | 86.69 $\pm$ 0.37                   | 31.8 $\pm$ 10.5 |
| sayplan      | 75.6 $\pm$ 2.1                   | 83.01 $\pm$ 0.42                   | 51.4 $\pm$ 27.6 |
| epog         | 60.2 $\pm$ 0.0                   | 84.51 $\pm$ 0.00                   | 13.9 $\pm$ 3.9  |
| gavel-map    | <b>92.6 <math>\pm</math> 1.0</b> | 82.45 $\pm$ 0.23                   | 39.3 $\pm$ 22.6 |
| gavel-static | <b>92.6 <math>\pm</math> 1.0</b> | 79.69 $\pm$ 0.45                   | 39.3 $\pm$ 22.6 |
| gavel        | <b>92.6 <math>\pm</math> 1.0</b> | <b>78.01 <math>\pm</math> 0.71</b> | 39.3 $\pm$ 22.6 |
| oracle       | 100.0 $\pm$ 0.0                  | 56.17 $\pm$ 0.00                   | —               |

**Graph-derived repair** addresses this limitation directly. Consider *gavel-basic*, a variant that only performs graph repairs and never re-queries the LLM. It already reaches 67.7% with Qwen3-4B and 76.2% with Qwen3-8B. On Qwen3-4B this exceeds feedback-only replanning by 12.3 points using 1 rather than 3.4 queries, and on Qwen3-8B it matches it at the same cost. When GAVEL additionally returns only the remaining semantic failures to the LLM, success rises to 88.8% and 91.8% with just 1.7 and 1.5 calls on average. The graph thus handles failures implied by action semantics, while LLM replanning is reserved for those that genuinely require semantic reasoning. For Qwen3-4B this also cuts planning time from 27.6 s under feedback-only replanning to 15.0 s. Fig. 3 shows the failure breakdown for a representative Qwen3-8B run, illustrating how errors are removed across the pipeline and which remain.

#### F. Experiment 2: Long-Horizon Multi-Task Execution

Having established reliable single-task planning, we next consider long-horizon instructions containing multiple tasks under partial observability. Here failures compound: although each component task is shorter than in Experiment 1 (4.86 versus 12.2 reference primitives), the complete instruction’s success is dependent on all component plans. As a result, *llm-only* completes just 19.9% instructions.

**Verification and repair** remain important at this scale. The SayPlan-style baseline, which returns graph-validation failures to the LLM but performs no graph-derived repair, reaches 75.6%; GAVEL completes 92.6%. GAVEL also reduces traveled distance from 83.01 m to 78.01 m on the matched sets both methods succeeded, and planning time by roughly 12 s per instruction. Repeatedly asking the LLM is therefore not equivalent to directly correcting failures the action model has already determined.

TABLE III  
EXPERIMENT 3: LOCAL AND HOSTED PLANNERS ON THE SAME 100-INSTRUCTION SUBSET.

| Planner         | Success (%)    |                                  | Time (s)        |                 |
|-----------------|----------------|----------------------------------|-----------------|-----------------|
|                 | LLM            | GAVEL                            | LLM             | GAVEL           |
| Qwen3-4B        | 2.4 $\pm$ 1.5  | <b>75.2 <math>\pm</math> 2.1</b> | 23.0 $\pm$ 7.4  | 35.0 $\pm$ 31.2 |
| Qwen3-8B        | 23.6 $\pm$ 1.5 | <b>89.2 <math>\pm</math> 1.3</b> | 31.8 $\pm$ 10.5 | 39.3 $\pm$ 22.6 |
| GPT-5.6 Sol     | 24.6 $\pm$ 3.9 | <b>99.2 <math>\pm</math> 0.8</b> | 16.5 $\pm$ 3.9  | 16.5 $\pm$ 4.0  |
| Claude Sonnet 5 | 38.6 $\pm$ 2.2 | <b>99.4 <math>\pm</math> 0.5</b> | 18.0 $\pm$ 4.4  | 18.2 $\pm$ 4.4  |

The EPoG-style baseline exposes the complementary limitation of *graph-only procedure generation*. It plans by transforming the believed graph toward the goal graph and succeeds on 60.2% of instructions. Its failures correspond exactly to the instructions requiring an appliance-induced semantic state (e.g., cooked, washed, or dried); “wash the plate” requires inferring that the dishwasher must be run, a procedure the graph-state difference alone cannot specify. This motivates keeping the LLM as the high-level semantic planner, while the graph world model determines if the LLM’s procedure is executable and repairs inconsistencies.

GAVEL’s own 38 residual failures on Experiment 2 lie mostly upstream of planning: 21 due to object extraction, with only 5 unexecutable plans, and 9 unmet goals.

Finally, we isolate GAVEL’s uncertainty reasoning. **Distributional beliefs** reduce mean distance from 82.45 m with *gavel-map* to 79.69 m with *gavel-static*, and **online reordering** after each completed task further reduces it to 78.01 m. Together these save 4.45 m, or  $\approx 5.4\%$ , while the ordering optimization itself takes only 17 ms per instruction.

#### G. Experiment 3: Does Model Scaling Replace GAVEL?

Finally, we ask if the above gains could instead be obtained by replacing compact local models with stronger LLMs. We evaluate all planners on the same 100-instruction subset of the multi-task benchmark, selecting every fifth instruction.

**Scaling alone** improves semantic planning but does not eliminate long-horizon failures. Without GAVEL, Qwen3-4B and Qwen3-8B solve only 2.4% and 23.6% of the subset, while GPT-5.6 Sol and Claude Sonnet 5 reach 24.6% and 38.6%. In contrast, Qwen3-4B with GAVEL reaches 75.2%, substantially outperforming the hosted models used alone. The gain from explicit world-model reasoning therefore seems to persist despite increasing LLM capability.

**Scaling and GAVEL are complementary.** With GAVEL, Qwen3-8B reaches 89.2% and both hosted models reach near-perfect success on this subset. Stronger LLMs reduce the remaining semantic planning errors, while GAVEL removes the action-level applicability and consistency failures that persist even for frontier models.

This is also inexpensive for the hosted models: planning time remains 16.5 s for GPT-5.6 Sol and changes only from 18.0 to 18.2 s for Claude Sonnet 5. The additional cost for compact models therefore comes from further LLM planning attempts rather than from graph verification and repair.

## VI. CONCLUSION

We presented GAVEL, a framework for long-horizon LLM planning built around an explicit graph world model representing object relations, action preconditions and effects, and beliefs over unobserved object locations. GAVEL rolls LLM-generated plans forward before execution, repairs failures whose corrections follow from the modeled action semantics, and reserves LLM queries for failures requiring semantic reasoning. Under partial observability, it uses distributional beliefs to estimate execution costs and reoptimizes the remaining task order as observations arrive.

Our results highlight the benefit of this division of responsibility. On BEHAVIOR-1K, graph-derived repair alone raises Qwen3-4B single-task success to 67.7%, exceeding feedback-only replanning. Allowing LLM re-queries for unresolved semantic failures further raises this to 88.8%. World-model reasoning is also complementary to scaling: Qwen3-4B with GAVEL outperforms stronger hosted planners when used alone, while both hosted models reach near-perfect success when combined with GAVEL. Retaining full object-location distributions and reoptimizing online further reduces traveled distance by approximately 5.4%.

Finally, our evaluation uses symbolic manipulation, so the graph captures logical action consequences but not geometric feasibility. Adding motion-level reachability and collision constraints would extend the same verify-and-repair mechanism to physical execution. The semantic prior is conditioned primarily on room type, and improving it based on repeated observations could capture environment-specific regularities. Finally, most residual failures now arise in grounding the instruction rather than planning over the graph, pointing to instruction understanding as the next bottleneck.

## REFERENCES

- [1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [2] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” *arXiv preprint arXiv:2207.05608*, 2022.
- [3] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *2023 IEEE Int. Conf. on robotics and automation (ICRA)*. IEEE, 2023, pp. 9493–9500.
- [4] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, “Llm-planner: Few-shot grounded planning for embodied agents with large language models,” in *IEEE/CVF Int. Conf. on computer vision*, 2023, pp. 2998–3009.
- [5] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, “On the planning abilities of large language models—a critical investigation,” *Advances in neural information processing systems*, vol. 36, pp. 75 993–76 005, 2023.
- [6] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldy, and A. Murthy, “Llms can’t plan, but can help planning in llm-modulo frameworks,” *arXiv preprint arXiv:2402.01817*, 2024.
- [7] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, “Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning,” *arXiv preprint arXiv:2307.06135*, 2023.
- [8] H. Yang, Z. Jiao, S. Wang, Y. Niu, S. Liu, and H. Liu, “Integrated exploration and sequential manipulation on scene graph with llm-based situated replanning,” *arXiv preprint arXiv:2602.04419*, 2026.
- [9] S. K. Ramakrishnan, D. S. Chaplot, Z. Al-Halah, J. Malik, and K. Grauman, “Poni: Potential functions for objectgoal navigation with interaction-free learning,” in *2022 CVPR*, 2022, pp. 18 868–18 878.
- [10] B. Yu, H. Kasaei, and M. Cao, “L3mvn: Leveraging large language models for visual target navigation,” in *2023 IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2023, pp. 3554–3560.
- [11] M. F. Ginting, S.-K. Kim, D. Fan, M. Palieri, M. Kochenderfer, and A. Agha-Mohammadi, “Seek: Semantic reasoning for object goal navigation in real world inspection tasks,” *arXiv:2405.09822*, 2024.
- [12] R. Wang, H.-L. Hsu, D. Hunt, J. Kim, S. Luo, and M. Pajic, “Comres-vm: Coordinated multi-robot exploration and search using vision language models,” *arXiv preprint arXiv:2509.26324*, 2025.
- [13] C. R. Garrett, C. Paxton, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox, “Online replanning in belief space for partially observable task and motion problems,” in *2020 ICRA*. IEEE, 2020, pp. 5678–5684.
- [14] A. Curtis, G. Matheos, N. Gothoskar, V. Mansinghka, J. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, “Partially observable task and motion planning with uncertainty and risk awareness,” *arXiv preprint arXiv:2403.10454*, 2024.
- [15] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martín-Martín, C. Wang, G. Levine, M. Lingelbach, J. Sun *et al.*, “Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation,” in *Conf. on Robot Learning*. PMLR, 2023, pp. 80–93.
- [16] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “Progprompt: Generating situated robot task plans using large language models,” *arXiv preprint arXiv:2209.11302*, 2022.
- [17] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “Llm+ p: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [18] A. Curtis, N. Kumar, J. Cao, T. Lozano-Pérez, and L. P. Kaelbling, “Trust the proc3s: Solving long-horizon robotics problems with llms and constraint satisfaction,” *arXiv preprint arXiv:2406.05572*, 2024.
- [19] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, “Text2motion: From natural language instructions to feasible plans,” *Autonomous Robots*, vol. 47, no. 8, pp. 1345–1365, 2023.
- [20] T. Feng, Y. Wu, G. Lin, and J. You, “Graph world model,” *arXiv preprint arXiv:2507.10539*, 2025.
- [21] P. Anokhin, N. Semenov, A. Sorokin, D. Evseev, A. Kravchenko, M. Burtsev, and E. Burnaev, “Arigraph: Learning knowledge graph world models with episodic memory for llm agents,” *arXiv preprint arXiv:2407.04363*, 2024.
- [22] C. Agia, K. M. Jatavallabhula, M. Khodeir, O. Miksik, V. Vineet, M. Mukadam, L. Paull, and F. Shkurti, “Taskography: Evaluating robot task planning over large 3d scene graphs,” in *Conf. on Robot Learning*. PMLR, 2022, pp. 46–58.
- [23] Z. Jiao, Y. Niu, Z. Zhang, S.-C. Zhu, Y. Zhu, and H. Liu, “Sequential manipulation planning on scene graph,” in *IROS*, 2022, pp. 8203–8210.
- [24] D. Ekpo, M. Levy, S. Suri, C. Huynh, A. Swaminathan, and A. Shrivastava, “Verigraph: Scene graphs for execution verifiable robot planning,” *arXiv preprint arXiv:2411.10446*, 2024.
- [25] A. O. Onishchenko, A. K. Kovalev, and A. I. Panov, “Lookplangraph: Embodied instruction following method with vlm graph augmentation,” *arXiv preprint arXiv:2512.21243*, 2025.
- [26] S. Amiri, K. Chandan, and S. Zhang, “Reasoning with scene graphs for robot planning under partial observability,” *IEEE Robotics and Automation Letters (RA-L)*, vol. 7, no. 2, pp. 5560–5567, 2022.
- [27] A. Rajvanshi, K. Sikka, X. Lin, B. Lee, H.-P. Chiu, and A. Velasquez, “Saynav: Grounding large language models for dynamic planning to navigation in new environments,” in *Proceedings of the Int. Conf. on Automated Planning and Scheduling*, vol. 34, 2024, pp. 464–474.
- [28] D. Honerkamp, M. Büchner, F. Despinoy, T. Welschhold, and A. Valada, “Language-grounded dynamic scene graphs for interactive object search with mobile manipulation,” *IEEE RA-L*, pp. 8298–8305, 2024.
- [29] Z. Wang, S. Cai, G. Chen, A. Liu, X. S. Ma, and Y. Liang, “Describe, explain, plan and select: interactive planning with llms enables open-world multi-task agents,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 153–34 189, 2023.